



CSE 3233

Computer Network Programming

Final Exam Project

OmniConnect Smart Home System

Elif Nida Solakoğlu

220316033

2025-Fall

OmniConnect Smart Home System

Network Programming Final Project

1. INTRODUCTION

Smart home systems have become one of the most practical applications of modern network programming and distributed systems. With the rapid growth of Internet of Things (IoT) technologies, multiple devices such as sensors, actuators, and user interfaces are required to communicate efficiently, reliably, and in real time. These systems must handle different types of data traffic, ranging from critical control commands to high-frequency telemetry data.

The OmniConnect Smart Home System was designed as a comprehensive network programming project that demonstrates the practical use of TCP, UDP, and inter-process communication techniques within a single integrated system. The primary objective of this project is to simulate a realistic smart home environment where multiple independent devices communicate through a centralized hub while maintaining responsiveness, reliability, and scalability.

In this project, TCP is used for reliable communication such as device registration, command transmission, and file transfer operations. UDP is used for time-sensitive telemetry data, where low latency is more important than guaranteed delivery. Additionally, local inter-process communication is implemented using Named Pipes to efficiently collect and store system logs without introducing unnecessary network overhead.

By combining these communication models, OmniConnect

provides a clear demonstration of how different networking techniques can be used together in a real-world application.

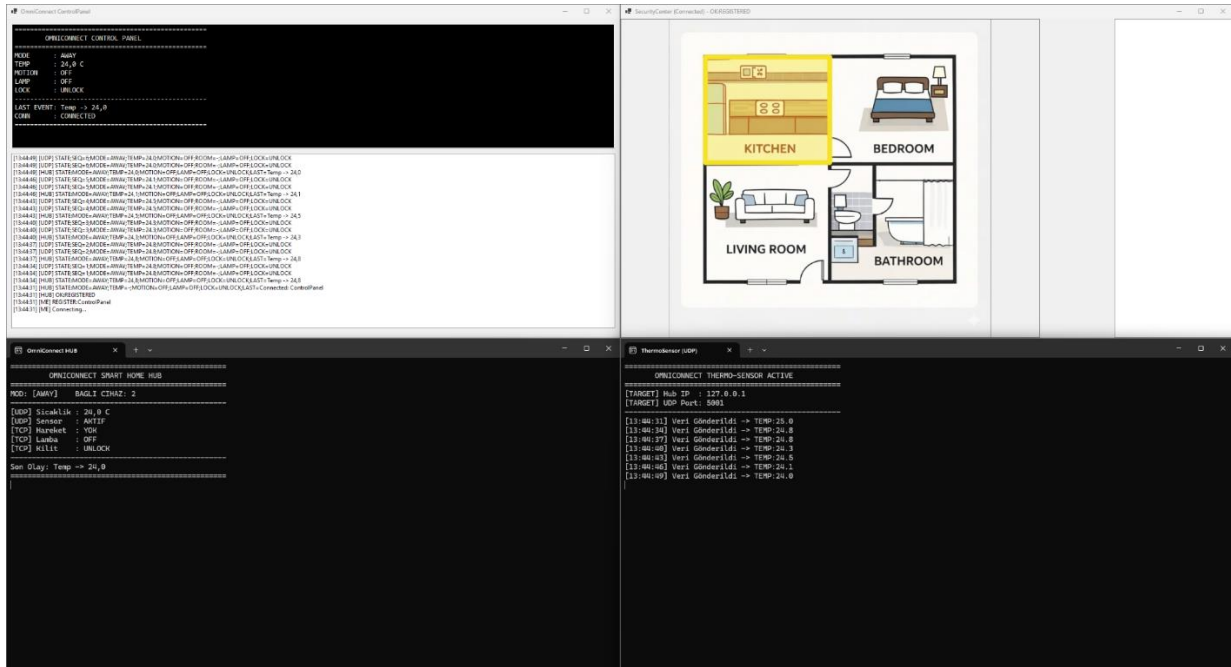
2. SYSTEM OVERVIEW

The OmniConnect system follows a centralized smart home architecture where all devices communicate through a single central component called the Hub. The Hub is responsible for managing device connections, processing incoming data, evaluating automation rules, and broadcasting system state updates to connected clients.

The system consists of the following main components:

- **Hub:** The central server responsible for managing all TCP and UDP communication.
- **ThermoSensor:** A UDP-based temperature sensor that periodically sends telemetry data.
- **MotionSensor:** A TCP-based sensor that reports motion events and transfers snapshot data.
- **SmartLamp:** A TCP-controlled actuator that responds to on/off commands.
- **SmartLock:** A TCP-controlled actuator responsible for locking and unlocking.
- **ControlPanel:** A monitoring client that displays real-time system state.
- **SecurityCenter:** A visualization client that displays alerts and emergency states.
- **LogCollector:** A background service that collects logs via Named Pipes.

All components are implemented as independent processes and can be started or stopped individually. This modular design allows the system to simulate real-world distributed behavior while remaining easy to test and extend.



(Overall system with all components running)

3. SYSTEM ARCHITECTURE

OmniConnect is built using a centralized hub-and-spoke architecture. In this model, all devices connect directly to the Hub rather than communicating with each other. This design simplifies coordination, state management, and rule evaluation.

The Hub uses a multi-threaded architecture to handle multiple simultaneous TCP clients. Each incoming TCP connection is processed asynchronously, ensuring that one slow or disconnected client does not block the entire system. Connected clients are stored in a thread-safe `ConcurrentDictionary`, which allows safe access from multiple threads without explicit locking at a global level.

UDP communication is handled independently through a dedicated listening loop. This separation ensures that high-frequency UDP packets do not interfere with TCP command handling.

The architecture also includes a local IPC layer for logging. Instead of sending logs over the network, the Hub communicates with the LogCollector using Named Pipes, which provides high performance and low latency for local communication.

This layered architecture clearly separates concerns and improves the maintainability and scalability of the system.

4. TCP COMMUNICATION DESIGN

TCP is used in OmniConnect for all operations that require reliable, ordered, and guaranteed delivery. These operations include device registration, command transmission, event reporting, and binary file transfer.

4.1 Device Registration

When a device connects to the Hub via TCP, it must first send a REGISTER message identifying its device type. The Hub validates this message and registers the client session internally. Once registration is complete, the device receives an acknowledgment message confirming successful registration.

This mechanism ensures that the Hub always knows the role of each connected client and can route commands accordingly.

```
ThermoSensor (UDP) X + v
=====
OMNICONNECT THERMO-SENSOR ACTIVE
=====
[TARGET] Hub IP : 127.0.0.1
[TARGET] UDP Port: 5001
-----
[13:45:37] Veri Gönderildi -> TEMP:24.8
[13:45:40] Veri Gönderildi -> TEMP:25.3
[13:45:43] Veri Gönderildi -> TEMP:24.9
[13:45:46] Veri Gönderildi -> TEMP:24.9
[13:45:49] Veri Gönderildi -> TEMP:24.6
[13:45:52] Veri Gönderildi -> TEMP:24.4
|
```

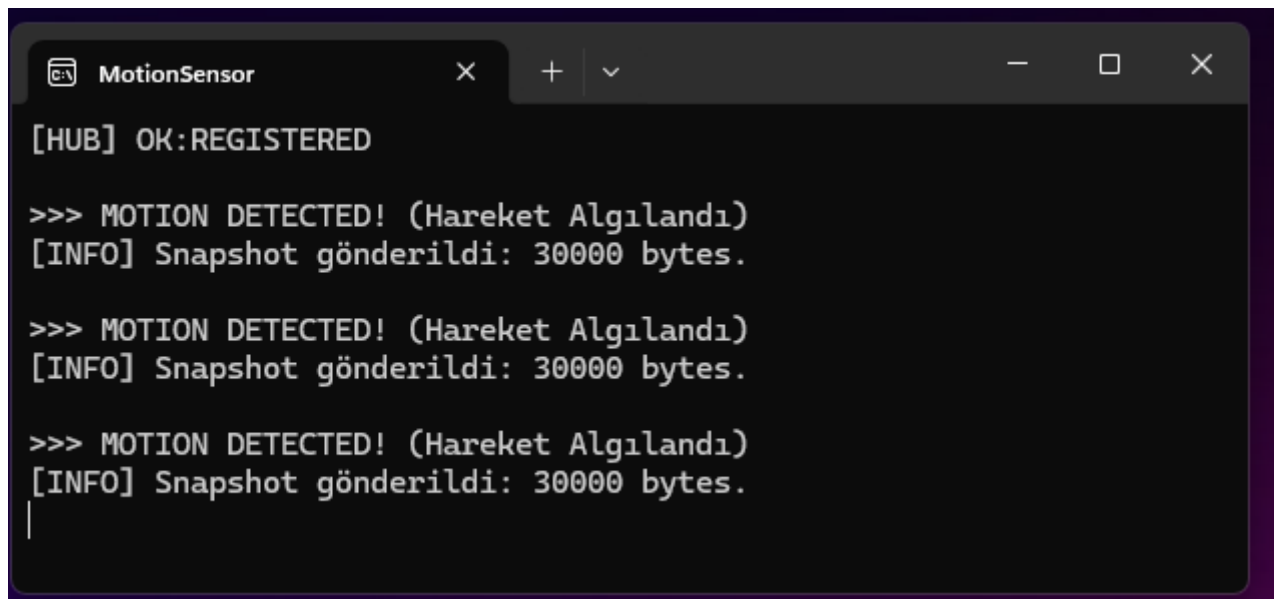
(Device registration messages in console)

4.2 Command Transmission

Commands such as turning the lamp on or locking the door are sent from the ControlPanel to the Hub via TCP. The Hub processes these commands and forwards them to the appropriate device. Each command is acknowledged to ensure synchronization between the sender and the receiver.

4.3 Motion Events and File Transfer

When motion is detected, the MotionSensor sends a TCP event message to the Hub. After the event is received, the MotionSensor initiates a binary file transfer to send a snapshot. The Hub first receives the file size and then reads the raw byte stream directly from the network stream.



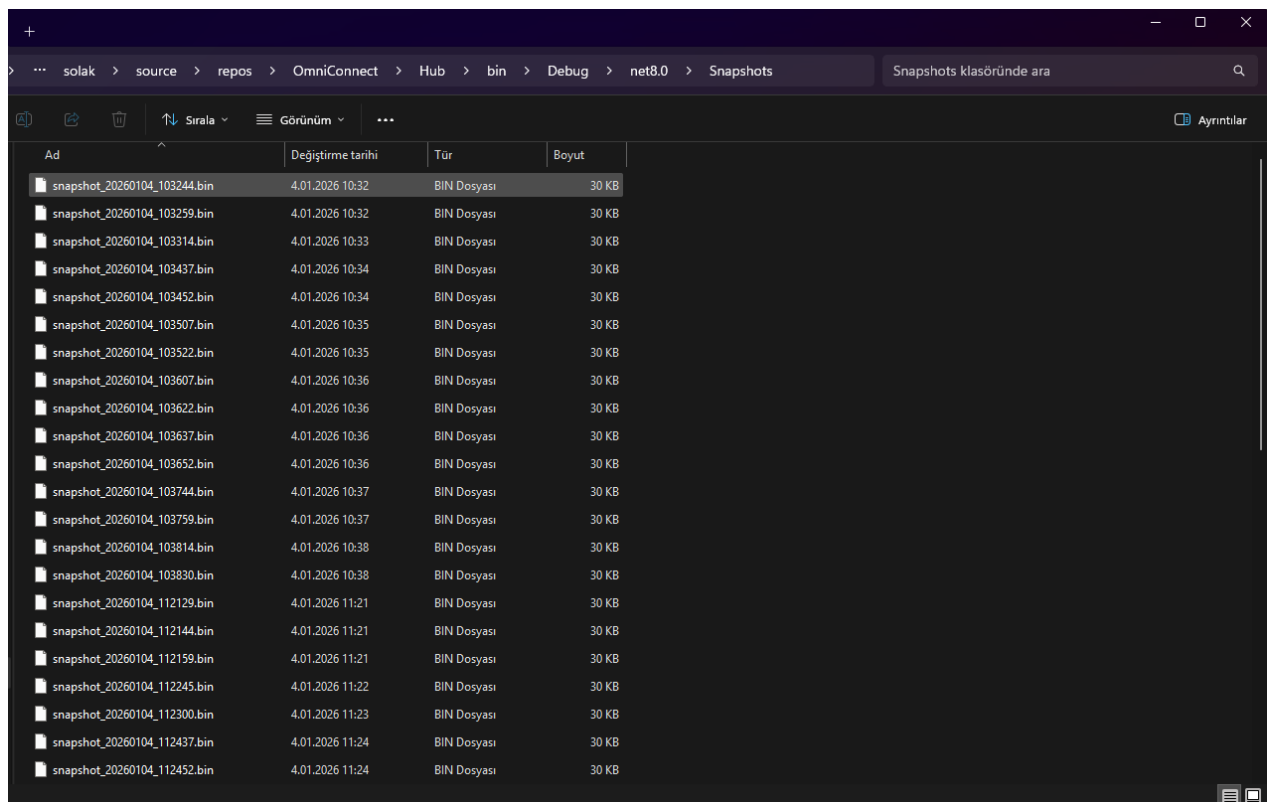
```
[HUB] OK:REGISTERED

>>> MOTION DETECTED! (Hareket Algılandı)
[INFO] Snapshot gönderildi: 30000 bytes.

>>> MOTION DETECTED! (Hareket Algılandı)
[INFO] Snapshot gönderildi: 30000 bytes.

>>> MOTION DETECTED! (Hareket Algılandı)
[INFO] Snapshot gönderildi: 30000 bytes.
```

(MotionSensor snapshot transfer console output)



Ad	Değiştirme tarihi	Tür	Boyut
snapshot_20260104_103244.bin	4.01.2026 10:32	BIN Dosyası	30 KB
snapshot_20260104_103259.bin	4.01.2026 10:32	BIN Dosyası	30 KB
snapshot_20260104_103314.bin	4.01.2026 10:33	BIN Dosyası	30 KB
snapshot_20260104_103437.bin	4.01.2026 10:34	BIN Dosyası	30 KB
snapshot_20260104_103452.bin	4.01.2026 10:34	BIN Dosyası	30 KB
snapshot_20260104_103507.bin	4.01.2026 10:35	BIN Dosyası	30 KB
snapshot_20260104_103522.bin	4.01.2026 10:35	BIN Dosyası	30 KB
snapshot_20260104_103607.bin	4.01.2026 10:36	BIN Dosyası	30 KB
snapshot_20260104_103622.bin	4.01.2026 10:36	BIN Dosyası	30 KB
snapshot_20260104_103637.bin	4.01.2026 10:36	BIN Dosyası	30 KB
snapshot_20260104_103652.bin	4.01.2026 10:36	BIN Dosyası	30 KB
snapshot_20260104_103744.bin	4.01.2026 10:37	BIN Dosyası	30 KB
snapshot_20260104_103759.bin	4.01.2026 10:37	BIN Dosyası	30 KB
snapshot_20260104_103814.bin	4.01.2026 10:38	BIN Dosyası	30 KB
snapshot_20260104_103830.bin	4.01.2026 10:38	BIN Dosyası	30 KB
snapshot_20260104_112129.bin	4.01.2026 11:21	BIN Dosyası	30 KB
snapshot_20260104_112144.bin	4.01.2026 11:21	BIN Dosyası	30 KB
snapshot_20260104_112159.bin	4.01.2026 11:21	BIN Dosyası	30 KB
snapshot_20260104_112245.bin	4.01.2026 11:22	BIN Dosyası	30 KB
snapshot_20260104_112300.bin	4.01.2026 11:23	BIN Dosyası	30 KB
snapshot_20260104_112437.bin	4.01.2026 11:24	BIN Dosyası	30 KB
snapshot_20260104_112452.bin	4.01.2026 11:24	BIN Dosyası	30 KB

(Snapshots folder showing saved binary files)

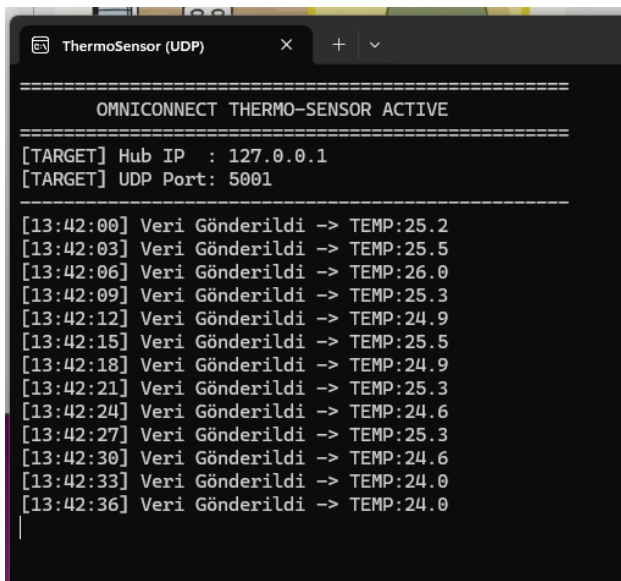
5. UDP COMMUNICATION DESIGN

UDP is used in OmniConnect for telemetry and real-time state broadcasting. Since temperature updates are sent frequently and

older values quickly become irrelevant, UDP is preferred to avoid unnecessary TCP overhead.

5.1 Temperature Telemetry

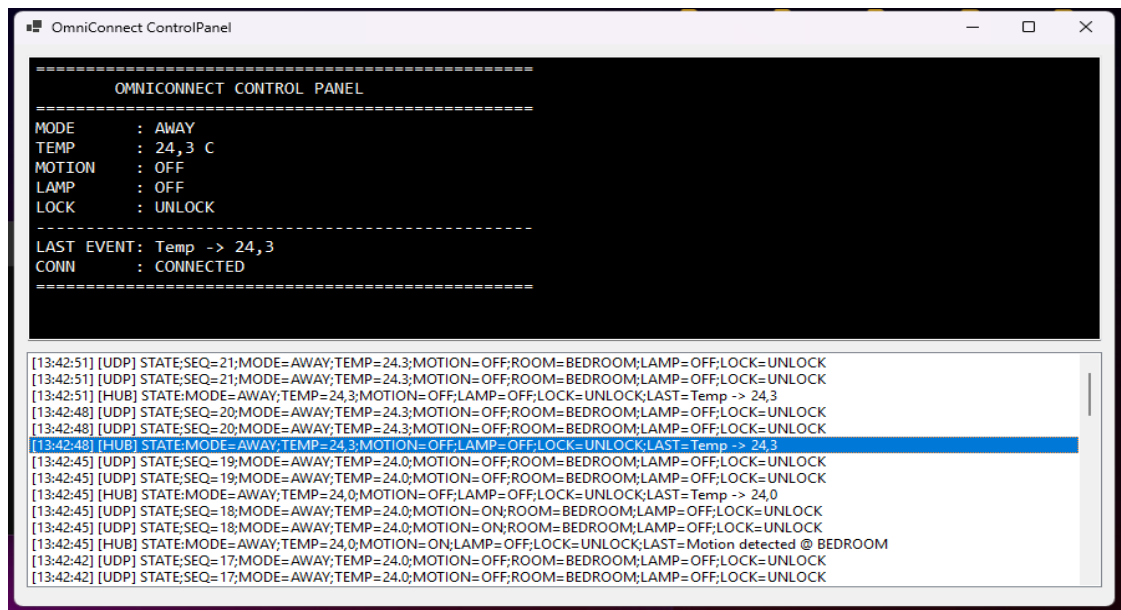
The ThermoSensor sends temperature readings to the Hub at fixed intervals using UDP. Values are formatted using invariant culture to avoid localization issues.



```
ThermoSensor (UDP)
=====
OMNICONNECT THERMO-SENSOR ACTIVE
=====
[TARGET] Hub IP : 127.0.0.1
[TARGET] UDP Port: 5001
=====
[13:42:00] Veri Gönderildi -> TEMP:25.2
[13:42:03] Veri Gönderildi -> TEMP:25.5
[13:42:06] Veri Gönderildi -> TEMP:26.0
[13:42:09] Veri Gönderildi -> TEMP:25.3
[13:42:12] Veri Gönderildi -> TEMP:24.9
[13:42:15] Veri Gönderildi -> TEMP:25.5
[13:42:18] Veri Gönderildi -> TEMP:24.9
[13:42:21] Veri Gönderildi -> TEMP:25.3
[13:42:24] Veri Gönderildi -> TEMP:24.6
[13:42:27] Veri Gönderildi -> TEMP:25.3
[13:42:30] Veri Gönderildi -> TEMP:24.6
[13:42:33] Veri Gönderildi -> TEMP:24.0
[13:42:36] Veri Gönderildi -> TEMP:24.0
```

5.2 State Broadcasting

After processing sensor data or commands, the Hub broadcasts the current system state via UDP. Each state packet includes a sequence number to help clients detect packet loss.



(UDP STATE packets with SEQ numbers in ControlPanel)

6. RULE ENGINE AND AUTOMATION

The Rule Engine is one of the most critical components of the OmniConnect system. It enables dynamic behavior without requiring code changes.

Rules are defined in an external rules.txt file using a simple IF-THEN syntax. The Hub continuously monitors this file and reloads it if changes are detected.

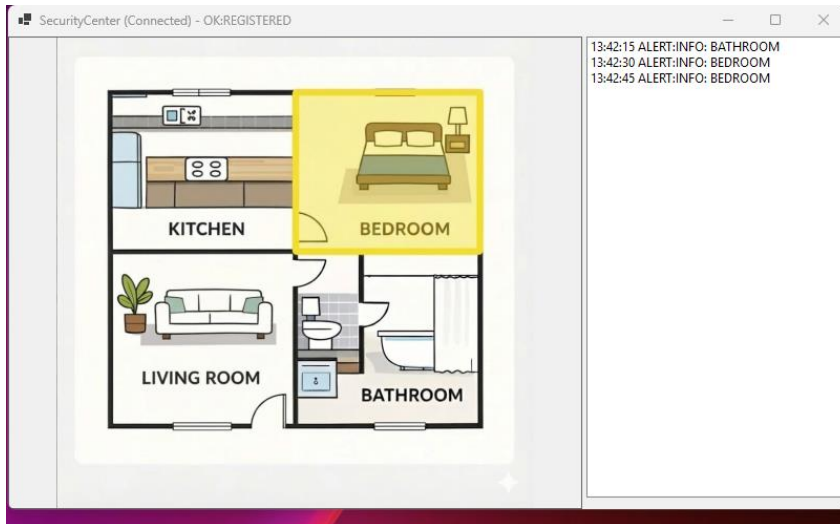
Supported conditions include:

- Temperature thresholds
- Motion detection
- System mode (HOME / AWAY)
- Room information
- Motion frequency within a time window

Actions include:

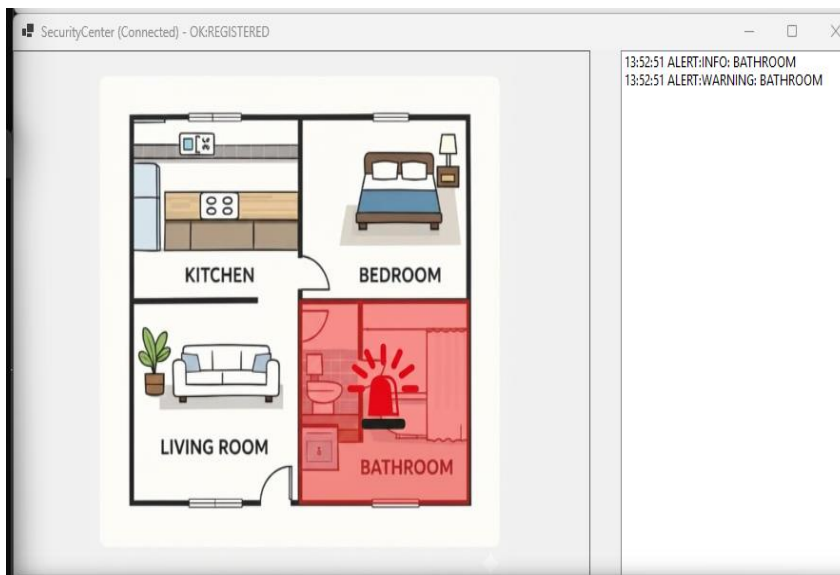
- Turning the lamp on or off
- Locking or unlocking doors
- Triggering alerts with different severity levels

6.1 INFO Alerts



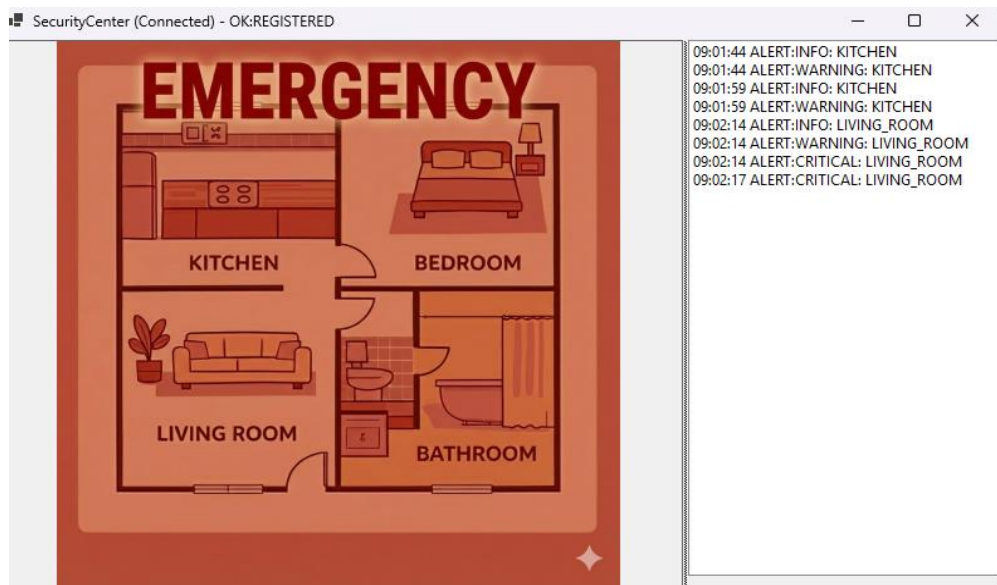
(INFO alert visualization)

6.2 WARNING Alerts



(WARNING alert visualization)

6.3 CRITICAL / EMERGENCY Alerts



(CRITICAL emergency state with automatic locking)

7. SECURITY CENTER VISUALIZATION

The SecurityCenter provides a visual representation of alerts generated by the Rule Engine. Different alert levels are displayed using distinct colors and icons to immediately convey severity to the user.

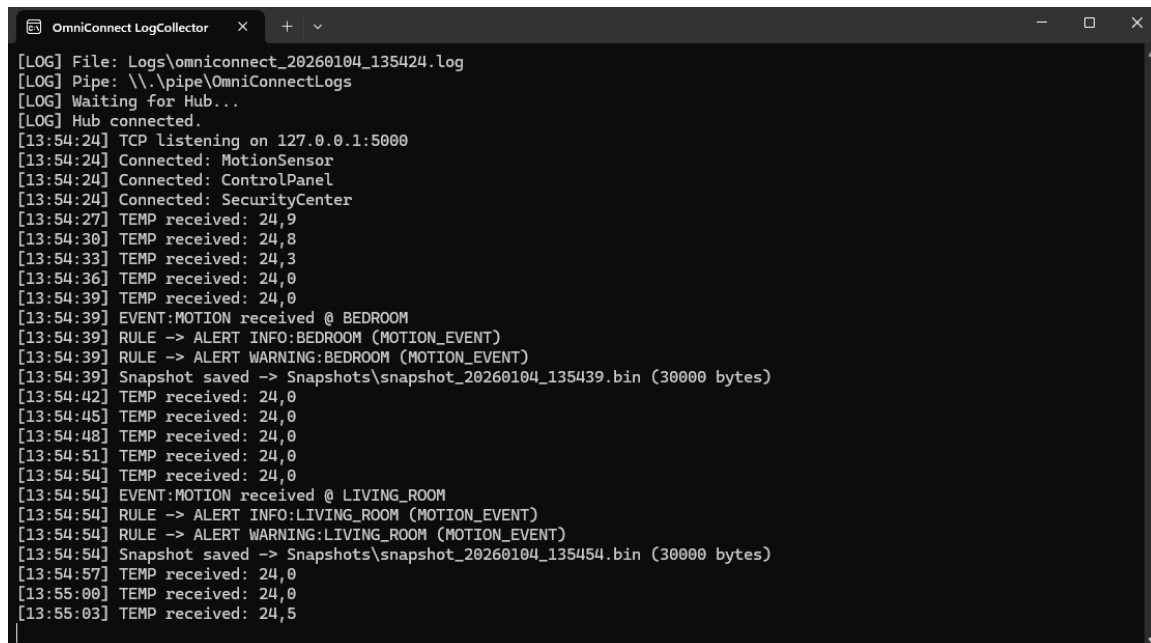
This component demonstrates how backend rule decisions can be translated into clear and intuitive user feedback.

8. INTER-PROCESS COMMUNICATION AND LOGGING

Instead of sending logs over the network, OmniConnect uses Named Pipes for local inter-process communication. The Hub writes logs to a Named Pipe, and the LogCollector reads and stores them asynchronously.

This design minimizes network usage and ensures that logging

never blocks core system functionality.



```
[LOG] File: Logs\omniconnect_20260104_135424.log
[LOG] Pipe: \\.\pipe\OmniConnectLogs
[LOG] Waiting for Hub...
[LOG] Hub connected.
[13:54:24] TCP listening on 127.0.0.1:5000
[13:54:24] Connected: MotionSensor
[13:54:24] Connected: ControlPanel
[13:54:24] Connected: SecurityCenter
[13:54:27] TEMP received: 24,9
[13:54:30] TEMP received: 24,8
[13:54:33] TEMP received: 24,3
[13:54:36] TEMP received: 24,0
[13:54:39] TEMP received: 24,0
[13:54:39] EVENT:MOTION received @ BEDROOM
[13:54:39] RULE -> ALERT INFO:BEDROOM (MOTION_EVENT)
[13:54:39] RULE -> ALERT WARNING:BEDROOM (MOTION_EVENT)
[13:54:39] Snapshot saved -> Snapshots\snapshot_20260104_135439.bin (30000 bytes)
[13:54:42] TEMP received: 24,0
[13:54:45] TEMP received: 24,0
[13:54:48] TEMP received: 24,0
[13:54:51] TEMP received: 24,0
[13:54:54] TEMP received: 24,0
[13:54:54] EVENT:MOTION received @ LIVING_ROOM
[13:54:54] RULE -> ALERT INFO:LIVING_ROOM (MOTION_EVENT)
[13:54:54] RULE -> ALERT WARNING:LIVING_ROOM (MOTION_EVENT)
[13:54:54] Snapshot saved -> Snapshots\snapshot_20260104_135454.bin (30000 bytes)
[13:54:57] TEMP received: 24,0
[13:55:00] TEMP received: 24,0
[13:55:03] TEMP received: 24,5
```

(LogCollector receiving logs via Named Pipes)

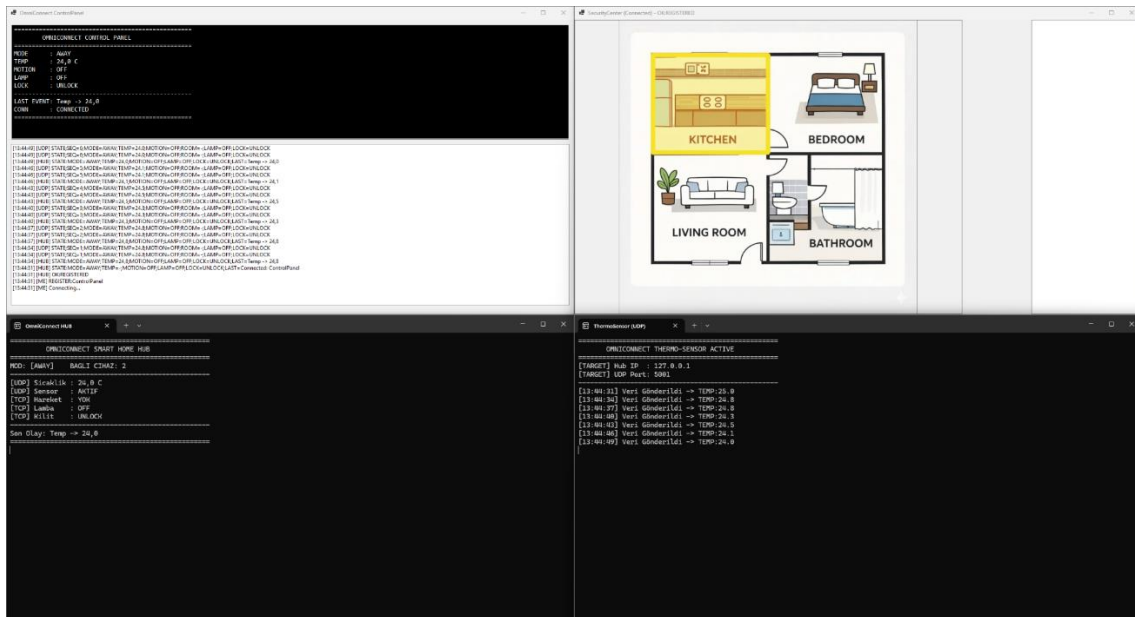
9. ERROR HANDLING AND ROBUSTNESS

All network operations are wrapped in try-catch blocks to ensure system stability. TCP disconnections are handled gracefully, and disconnected clients are safely removed from internal data structures.

UDP packet loss is tolerated by design, and system state broadcasts ensure eventual consistency.

10. TESTING AND VALIDATION

The system was tested with multiple simultaneous clients, frequent sensor updates, and repeated motion events. Rule switching during runtime was also tested to verify dynamic behavior without restarting the system.



11. CONCLUSION AND FUTURE WORK

This project successfully demonstrates the integration of TCP, UDP, and IPC mechanisms within a realistic smart home simulation. Through OmniConnect, practical experience was gained in multi-client server design, real-time data handling, and automation logic.

Future improvements could include persistent state storage, encrypted communication, and a web-based control interface.

12. AI TOOL USAGE AND REFLECTION

During the development of OmniConnect, I used multiple AI tools in a controlled and transparent way to accelerate implementation, improve quality, and ensure alignment with the course requirements.

Gemini was primarily used for rapid visual asset generation. In particular, I generated multiple map and alert visuals (e.g., INFO/WARNING/CRITICAL map variants) to make the

SecurityCenter interface more realistic and presentation-ready. This significantly improved the clarity of the severity concept during the demo and strengthened the project's user-facing output.

ChatGPT was used extensively for architecture planning, protocol design, and implementation-level guidance. Examples include: designing clean TCP/UDP message formats, structuring the Hub's async multi-client handling, implementing UDP state broadcasting with sequence numbers (SEQ) for packet-loss awareness, and establishing a consistent rules.txt grammar for the Rule Engine. Suggestions were always verified by running the system and confirming expected behaviors using logs, screenshots, and repeated test scenarios.

Claude was used as a verification-oriented assistant. I used it to compare my project's feature set against the instructor's project template and grading rubric (TCP server/client usage, UDP broadcasting, concurrency, robustness, and completeness). This helped ensure the project did not drift away from the assignment expectations and that the final demo scenarios (INFO / WARNING / CRITICAL) were defensible and clearly connected to the rubric categories.

Overall, these tools were used to support understanding and productivity—not to replace comprehension. Every major change was validated through local testing, log inspection, and reproducible demo scenarios.